# LMAT1351 - Approximation : méthodes et théorie
# Projet 2023

Straet Pierre

Décembre 2023

# 1 Interpolation polynomiale

## 1.1 Définition de la fonction

La fonction implémente l'algorithme vu au cours se basant sur les polynômes de Newton, en voici les premières équations :

$$
\begin{aligned}
&f(x_0) = p_n(x_0) = C_0 \\
&f(x_1) = p_n(x_1) = C_0 + C_1(x_1 - x_0) \\
&f(x_2) = p_n(x_2) = C_0 + C_1(x_2 - x_0) + C_2(x_2 - x_0)(x_2 - x_1) \\
&... \\
&f(x_n) = p_n(x_n) = C_0 + C_1(x_n - x_0) + ... + C_n(x_n - x_0)...(x_n - x_{n-1})
\end{aligned}
$$

Nous pouvons en effet pour chacune d'entre elles isoler le coefficient $C_i$ et ainsi déterminer le polynôme de degré n par récurrence.

## 1.2 Graphes

### 1.2.1 L'exponentielle

Voici les interpolations polynomiales sur l'intervalle $[-1, 1]$ de l'exponentielle construites à partir de 1 à 5 noeuds équidistants. Ceux-ci correspondent aux polynômes de degrés 0 à 4.

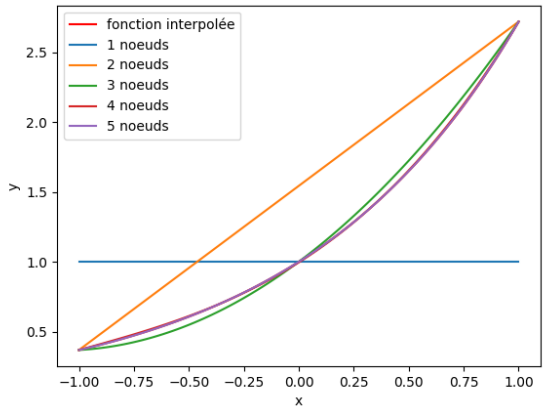


FIGURE 1 – Interpolation polynomiale de l'exponentielle

Observons en premier lieu que plus le degré du polynôme d'interpolation augmente, plus l'interpolation semble se rapprocher de la courbe initiale. Nous pouvons en effet noter qu'à partir du degré 3, nous sommes incapables de faire la distinction entre la fonction exponentielle et ses interpolations. Il est important de garder à l'esprit que ces observations se font sur l'intervalle $[-1, 1]$ et que nous ne pouvons pas les extrapoler à tout $\mathbb{R}$.

Comparons à présent ces résultats avec la théorie. Nous allons utiliser le Théorème du reste de Lagrange pour vérifier si nos interpolations se rapprochent ou non de la borne donnée dans ce théorème. Ce dernier s'énonce comme suit,

Si $f \in C^{n+1}[a, b]$, alors pour les noeuds $x_i (i = 0, ..., n)$

$$|f(x) - L_n[f](x)| \leq \frac{1}{(n+1)!} \|f^{(n+1)}\|_\infty |W_{n+1}(x)| \tag{1}$$

où $W_{n+1}(x) = \prod_{i=0}^{n}(x - x_i)$.

Il est aisé d'implémenter les deux membres de l'inégalité d'autant que celui de droite ne demande que de dériver $e^x$, ce qui est facilement réalisable. Comme nous considérons les fonctions sur l'intervalle $[-1, 1]$, nous avons que $\|(e^x)^{(n+1)}\|_\infty = e$. Voici le graphe reprenant les 2 fonctions établies pour une interpolation à 5 noeuds :

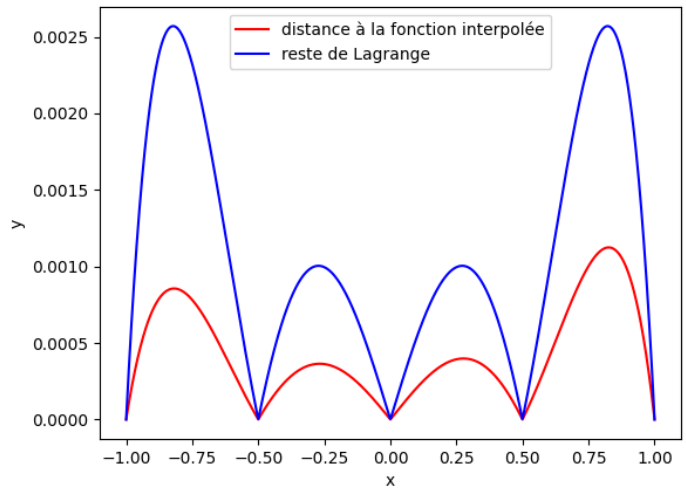


FIGURE 2 – Comparaison de l'erreur d'interpolation avec le reste de Lagrange

Nous pouvons donc voir sur la figure 2 que l'interpolation est relativement bonne, celle-ci étant bornée théoriquement aux alentours de 0,0025 et factuellement par un peu plus de 0,001. Nous pouvons également y observer que les racines des courbes se situent aux noeuds choisis, ce qui n'a rien d'étonnant étant donné que les polynômes y prennent la valeur de la fonction.

### 1.2.2 Le quotient

Nous étudions ici la fonction quotient définie sur l'intervalle $[-1, 1]$ comme :

$$f(x) = \frac{1}{1 + 25x^2}$$

Voici l'interpolation de cette fonction pour un petit nombre ($n \leq 7$) de noeuds ainsi que pour plus un grand ($n = 16$).

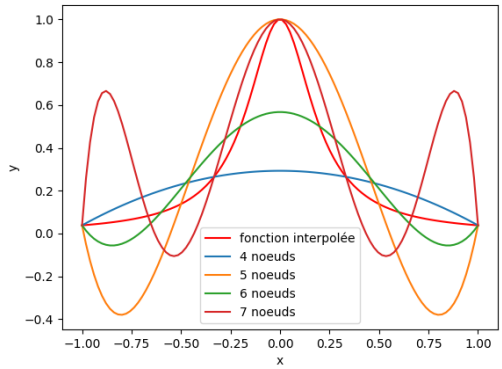


FIGURE 3 – Interpolation de la fonction pour peu de nœuds

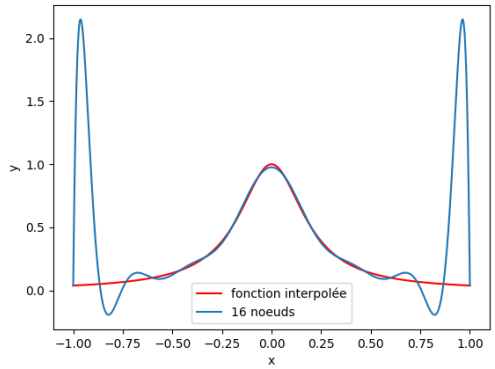


FIGURE 4 – Interpolation de la fonction pour beaucoup de nœuds

Sur ces figures, il est intéressant de remarquer la difficulté qu'ont les polynômes à interpoler la fonction donnée. Il est également intrigant de voir que plus le degré augmente plus l'interpolation sera meilleure au centre mais mauvaise sur les bords. Notons aussi que pour les polynômes construits avec peu de noeuds, il est avantageux d'en avoir un nombre impair. En effet, ceci les oblige à passer par le maximum de la fonction et rend le tout légèrement meilleur. Par ailleurs, pour les interpolations de 5 et 7 noeuds, nous constatons tout de même un début de la dégénérescence observée sur les polynômes de grand degré.

Concernant la borne donnée par Lagrange (voir (1)), nous pouvons voir que celle-ci est loin d'être atteinte.

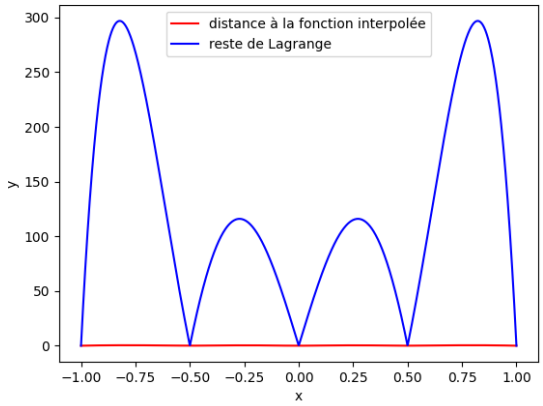


FIGURE 5 – Comparaison de l'erreur d'interpolation avec le reste de Lagrange

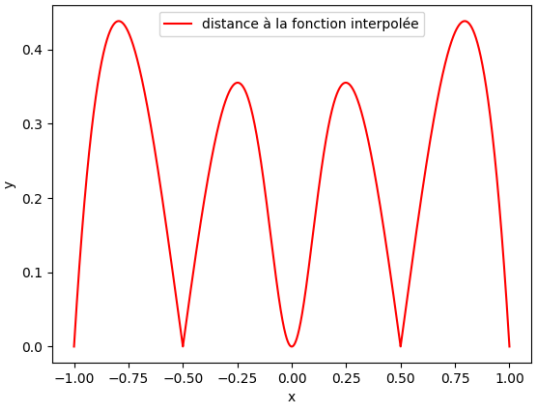


FIGURE 6 – Distance de l'interpolation au quotient

Ces graphes réalisés pour l'interpolation de 5 noeuds nous montrent en effet que l'erreur obtenue est majorée par 0.5, ce qui est dérisoire vis à vis du reste calculé qui lui est borné par 300. C'est en quelque sorte le phénomène de Runge. Nous en déduisons que le reste de Lagrange n'est pas toujours très restrictif et permet des erreurs assez conséquentes.

### 1.2.3 La valeur absolue

Nous allons finir cette section avec l'étude de la fonction valeur absolue.

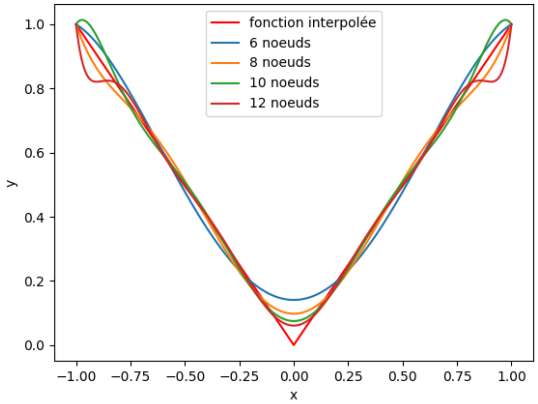


FIGURE 7 – Interpolation avec de petits nombres pairs de noeuds

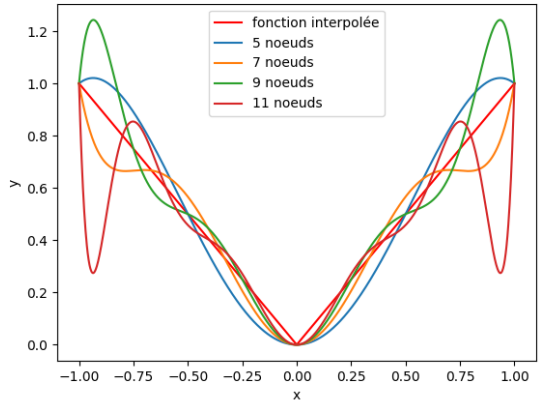


FIGURE 8 – Interpolation avec de petits nombres impairs de noeuds

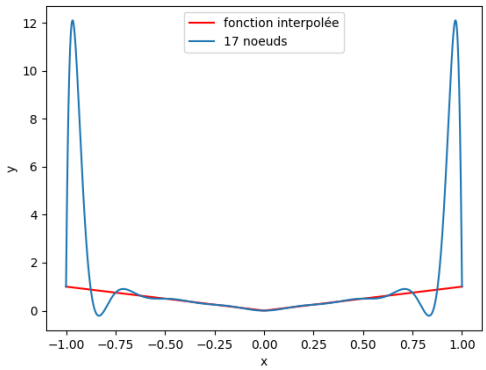


FIGURE 9 – Interpolation avec beaucoup de noeuds

Similairement à la fonction précédente, une première chose que nous pouvons observer est que les polynômes de grand degré ont du mal à interpréter correctement le bord de la fonction. Au vu du cours, ceci n'est peut-être pas si étonnant. En effet, nous avons vu que les noeuds qui minimisent l'erreur de l'interpolation sont ceux donnés par Chebyshev, qui en concentrent plus sur les bords de l'intervalle. Ceci permet alors de meilleures interprétations sur l'extérieur du graphe à l'instar des noeuds équidistants.

Par contre, contrairement au quotient, la valeur absolue se laisse plutôt facilement approximer par les polynômes de petits degrés. Nous pouvons également noter quelque chose d'intéressant : l'erreur de l'interpolation varie en fonction de la parité du nombre de noeuds. Celle-ci augmente pour les impairs et diminue pour les pairs, cela signifie que les polynômes de degrés impairs ont une meilleure interprétation de la valeur absolue.

Pour ce qui est de la comparaison avec le reste de Lagrange (voir (1)), celui-ci ne s'applique pas à la fonction valeur absolue étant donné que cette dernière n'appartient pas à $C^{(n+1)}[-1,1]$.

# 3 Splines d'interpolation

## 3.1 Définition de la fonction

Les splines cubiques d'interpolation étant de la forme, pour $x_{j-1} \leq x \leq x_j$ et $j \in \{1, n+1\}$ :

$$S(x) = \frac{(x_j - x)^3}{6(x_j - x_{j-1})} z_{j-1} + \frac{(x - x_{j-1})^3}{6(x_j - x_{j-1})} z_j + c_j(x - x_{j-1}) + d_j(x_j - x) \tag{2}$$

nous devons en calculer les $z_j$, $c_j$ et $d_j$, ce qui nous donne $3n + 4$ inconnues. On cherche donc autant d'équations afin d'en obtenir une solution unique.

Nous pouvons d'ores et déjà en établir $2n + 2$ en demandant que l'approche inférieure et supérieure du noeud de notre spline égale son évaluation en la fonction interpolée, c'est à dire

$$\begin{cases} S_+(x_j) = f(x_j) & \forall j \in \{1, n\} \\ S_-(x_j) = f(x_j) & \forall j \in \{1, n\} \\ S_+(x_0) = f(x_0) \\ S_-(x_{n+1}) = f(x_{n+1}). \end{cases}$$

Ensuite, nous rajouterons des conditions sur la continuité de S' pour en obtenir n de plus. On a les équations :

$$S'_+(x_j) = S'_-(x_j) \qquad \forall j \in \{1, n\}.$$

Jusqu'alors, il nous manque 2 équations pour arriver aux $3n+4$. Celles-ci nous sont données par les conditions aux bords de la dérivée seconde. En effet, comme nous voulons implémenter des splines naturelles, on aura

$$z_0 = 0 = z_{n+1}.$$

Il ne reste plus qu'à mettre en équations et résoudre le système pour trouver la spline cubique d'interpolation donnée en (2).

## 3.2 Graphes

### 3.2.1 L'exponentielle

Vous trouverez plus bas les splines d'interpolation de la fonction $e^x$ pour 3, 4 et 5 noeuds équidistants.

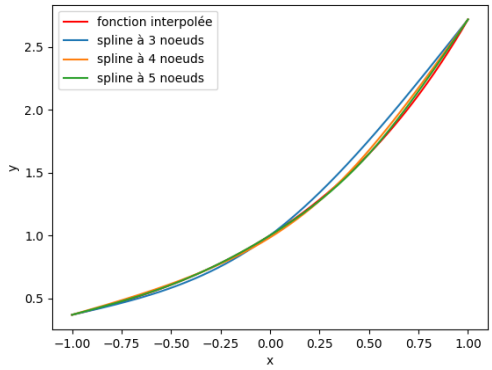


FIGURE 10 – Splines d'interpolation de l'exponentielle

Nous pouvons de nouveau constater que l'exponentielle s'interprète facilement. En effet, nous ne voyons ici que les splines pour 3, 4 et 5 noeuds mais celles-ci épousent déjà correctement la courbe. Nous n'avons pas besoin de beaucoup de noeuds pour avoir de belles interpolations. Ceci n'est pas tellement surprenant, car comme nous l'avions déjà remarqué lors de l'interpolation polynomiale, les polynômes de degré 3 sont suffisants pour approcher l'exponentielle.

### 3.2.2 Le quotient

Pour l'étude de cette fonction, le quotient étant défini plus haut dans la section 1 Interpolation polynomiale, nous avons les graphes des splines d'interpolation construites sur base de 4 à 7 noeuds et ensuite de 15 et 16 noeuds.

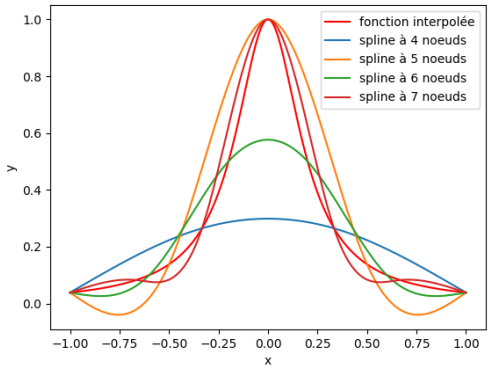


FIGURE 11 – Splines pour 4, 5 et 6 noeuds de la fonction quotient

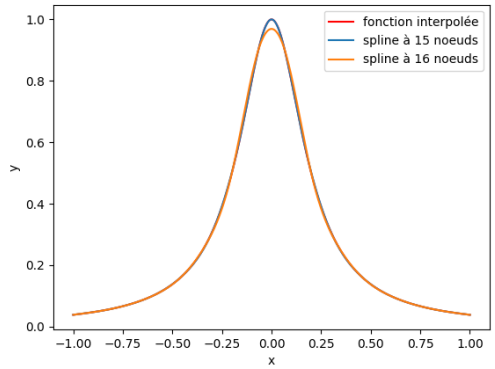


FIGURE 12 – Splines pour 15 et 17 noeuds de la fonction quotient

Nous pouvons alors y observer tout d'abord qu'en général les splines offrent une bien meilleure interprétation de la fonction qu'une interpolation polynomiale. En effet, à l'instar de cette dernière, augmenter le nombre de noeuds ne crée pas de dégénérescence sur les bords, nous ne savons même pas distinguer la spline du graphe initial pour un degré impair. Par ailleurs, on constate une similitude entre cette dernière au degré 3 et la spline cubique de 4 noeuds.

Enfin, notons que les splines construites avec un nombre pair de noeuds ont plus de difficultés à correspondre à la fonction initiale contrairement aux nombres impairs qui permettent de passer par le sommet et donner correctement l'allure de la courbe.

### 3.2.3 La valeur absolue

Nous finissons l'étude des splines d'interpolation avec la fonction valeur absolue. En voici les graphes pour 4, 5 et 6 noeuds ainsi que pour 15 et 16 noeuds.

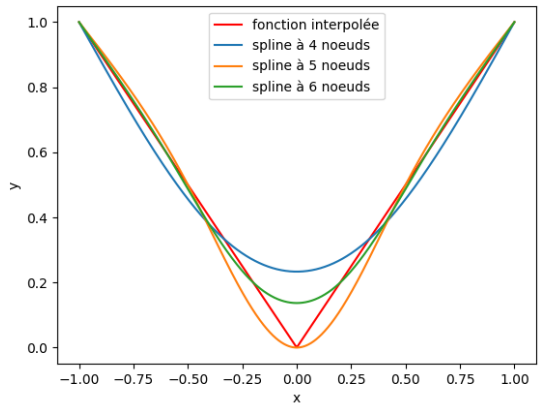


FIGURE 13 – Splines pour 4, 5 et 6 noeuds de la fonction valeur absolue

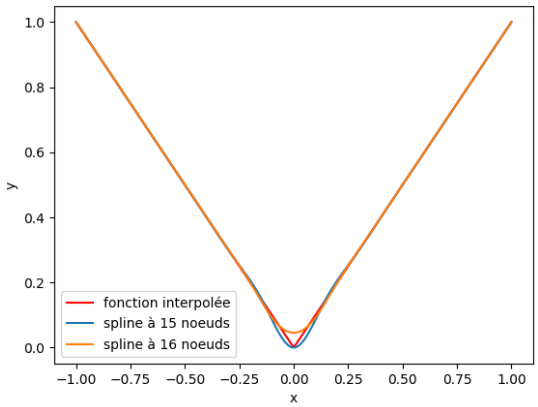


FIGURE 14 – Splines pour 15 et 17 noeuds de la fonction valeur absolue

Nous pouvons réitérer en partie l'analyse faite pour la fonction quotient. En effet, on observe que les splines restent meilleures également pour la valeur absolue. De même pour la différence entre les noeuds pairs et impairs, ainsi que pour l'absence de dégénérescences aux bords avec l'augmentation du nombre de noeuds. Notons en plus toutefois que le bas de la fonction reste difficile à interpréter. Cela vient du fait que la dérivée n'y est pas continue à l'inverse de nos splines qui y sont forcées.

# 4 Polynômes orthogonaux

## 4.1 Calculs des polynômes orthogonaux

Les polynômes orthogonaux de Legendre sont définis avec la fonction de poids suivante :

$$w_L(x) = (1-x)^\alpha (1+x)^\beta \mathbb{1}_{(-1,1)}(x) \quad avec \quad \alpha = \beta = 0.$$

C'est-à-dire,

$$w_L(x) = \mathbb{1}_{(-1,1)}(x).$$

Nous considérerons seulement les polynômes tels que $L(1) = 1$. Comme nous aurons régulièrement besoin de ceux-ci dans la suite, en voici l'implémentation.

Par définition des polynômes orthogonaux, on veut que

$$\langle L_i, L_j \rangle_{w_L} = 0 \quad \forall i \neq j \text{ dans } \{0, n\}$$

Ainsi, pour tout polynôme $p_i$, on a le système d'équations

$$\begin{cases} \int_{\mathbb{R}} L_i(x) L_j(x) w_L(x) dx = 0 & \forall 0 \leq j < i \\ L_i(1) = 1 \end{cases}$$

Étant donné que les intégrales portent sur des polynômes restreints à l'intervalle $[-1, 1]$, nous pouvons les calculer explicitement pour en sortir les coefficients de $L_i$, un polynôme de degré $i$, et en obtenir un simple système linéaire. En posant $L_0 = 1$, afin de satisfaire nos conditions initiales, il nous reste à procéder par récurrence pour trouver $L_n$.

## 4.2 Coefficients de récurrence

Les coefficients de récurrence sont utiles à plusieurs effets. Nous les retrouvons dans la construction des polynômes orthogonaux (voir sous-section suivante), ainsi que dans l'intégration par quadrature pour le calcul des poids de Christoffel (voir section suivante).

Pour les calculer, il existe plusieurs méthodes, par la formule de récurrence ou bien par le calcul de produits scalaires. C'est cette dernière méthode que nous allons détailler. Bien que ces coefficients permettent de trouver les polynômes orthogonaux, cette relation est réciproque, c'est-à-dire que nous pouvons également utiliser ces polynômes pour obtenir les coefficients. En voici la démarche pour ceux construits sur base du poids de Legendre :

On sait que comme n polynômes orthogonaux forment une base pour tout polynômes de degré $\leq$ n, on a

$$xL_n(x) = C_0 L_O(x) + C_1 L_1(x) + ... + C_{n+1} L_{n+1}(x)$$

Pour trouver les $C_j \forall j \in \{0, n+1\}$, prenons les produits scalaires avec $L_j \forall j \in \{0, n+1\}$ des 2 côtés de l'équation afin de ne garder que $C_j \langle L_j, L_j \rangle$ grâce à l'orthogonalité des polynômes. On trouve alors

$$\langle xL_n, L_j \rangle = C_j \langle L_j, L_j \rangle.$$

Et ce pour tout $j \in \{0, n+1\}$, on peut donc isoler les $C_j$ et écrire

$$xL_n = \frac{\langle xL_n, L_0 \rangle}{\langle L_0, L_0 \rangle} L_0 + ... + \frac{\langle xL_n, L_{n+1} \rangle}{\langle L_{n+1}, L_{n+1} \rangle} L_{n+1}.$$

Dès lors, de nouveau par l'orthogonalité des polynômes, on sait que $\langle xL_n, L_j \rangle = 0 \quad \forall j < n-1$. Finalement on a l'égalité

$$xL_n = \frac{\langle xL_n, L_{n+1} \rangle}{\langle L_{n+1}, L_{n+1} \rangle} L_{n+1} + \frac{\langle xL_n, L_n \rangle}{\langle L_n, L_n \rangle} L_n + \frac{\langle xL_n, L_{n-1} \rangle}{\langle L_{n-1}, L_{n-1} \rangle} L_{n-1}.$$

Les coefficients sont alors définis comme suit :

$$\begin{aligned}
\alpha_n &= \frac{\langle xL_n, L_{n+1} \rangle}{\langle L_{n+1}, L_{n+1} \rangle} \\
\beta_n &= \frac{\langle xL_n, L_n \rangle}{\langle L_n, L_n \rangle} \\
\gamma_n &= \frac{\langle xL_n, L_{n-1} \rangle}{\langle L_{n-1}, L_{n-1} \rangle}.
\end{aligned}$$

Et dans le cas normé, on sait en plus que $\langle L_{n-1}, L_{n-1} \rangle = 1$ ce qui nous donne :

$$\begin{aligned}
\alpha_n &= \langle xL_n, L_{n+1} \rangle \\
\beta_n &= \langle xL_n, L_n \rangle \\
\gamma_n &= \langle xL_n, L_{n-1} \rangle.
\end{aligned}$$

Notons par ailleurs que dans le cas normé, $\langle xL_n, L_{n+1} \rangle = \langle xL_{n+1}, L_n \rangle = \gamma_{n+1}$.

## 4.3 Construction des polynômes grâce aux coefficients de récurrence

Comme annoncé dans la sous-section précédente, il est possible de construire les polynômes orthogonaux grâce aux coefficients de récurrence. En effet, dans cette même section nous avions

$$xL_n = \alpha_n L_{n+1} + \beta_n L_n + \gamma_n L_{n-1}.$$

On isolant $L_{n+1}$ dans l'équation, on trouve la formule de récurrence qui nous permet de calculer $L_{n+1}$ en fonction des coefficients de récurrence et des polynômes $L_n$ et $L_{n-1}$ :

$$L_{n+1} = \frac{1}{\alpha_n}((x - \beta_n)L_n - \gamma_n L_{n-1}).$$

En prenant comme convention $L_{-1} = 0$ ainsi que $L_0 = 1$, nous avons notre formule complète.

## 4.4 Graphes

Voici quelques graphes des polynômes orthogonaux de Legendre tels que $L(1) = 1$ :

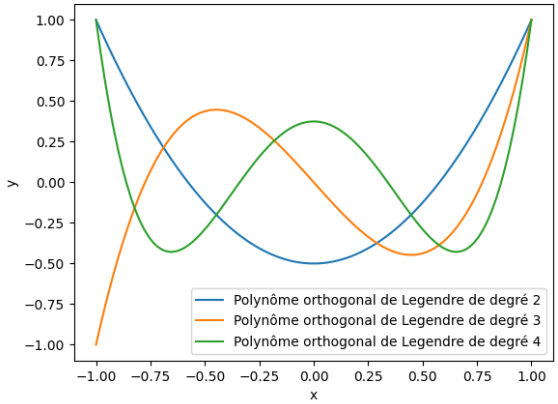


FIGURE 15 – Polynômes orthogonaux de Legendre de degré 2, 3, et 4

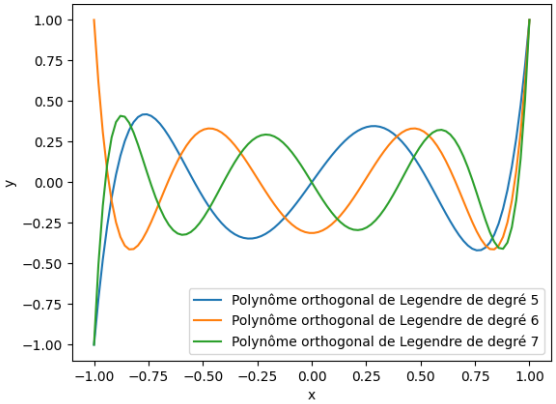


FIGURE 16 – Polynômes orthogonaux de Legendre de degré 5, 6 et 7

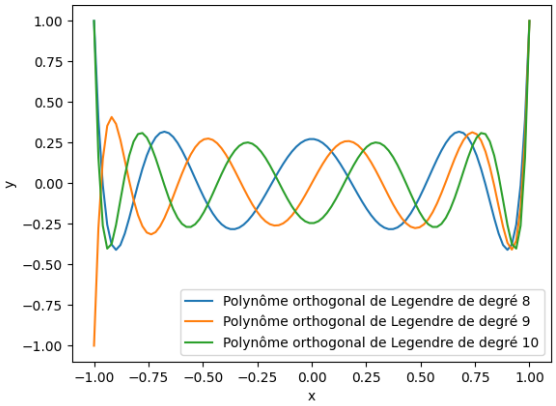


FIGURE 17 – Polynômes orthogonaux de Legendre de 8, 9 et 10 degrés

## 4.5 Racines des polynômes orthogonaux de Legendre

Dans l'optique d'analyser les racines des polynômes orthogonaux de Legendre, vous trouverez plus bas une représentation graphique de ceux-ci pour les degrés 3 et 15.

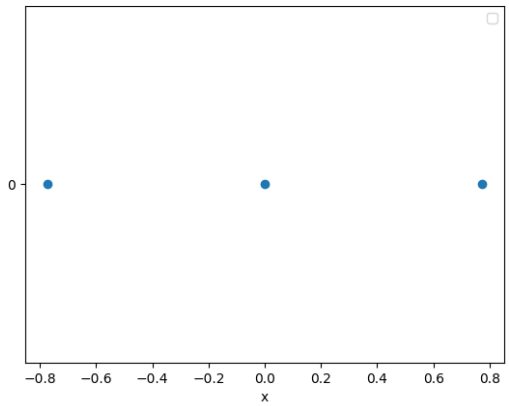


FIGURE 18 – Racines du polynômes orthogonal de Legendre de degré 3

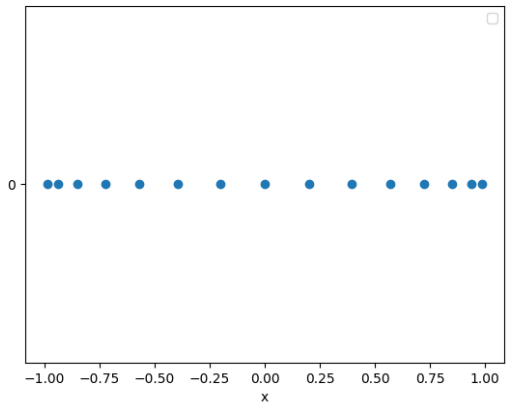


FIGURE 19 – Racines du polynômes orthogonal de Legendre de degré 15

Observons une première fois la dispersion des racines. Celles-ci se concentrent plus sur les extrémités de l'intervalle $[-1, 1]$. Cela peut nous faire penser aux noeuds de Chebyshev. Ensuite, comparons ces racines avec celles des polynômes du degré supérieur, c'est à dire celles des polynômes de degré 4 et 16.

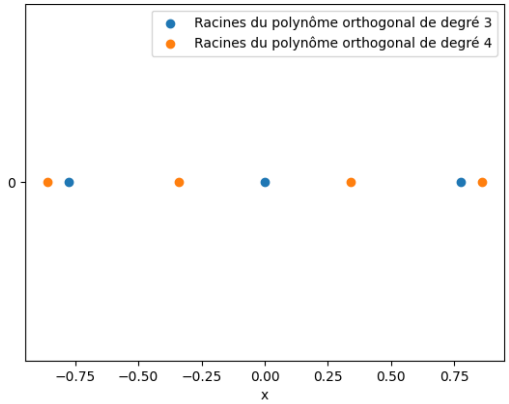


FIGURE 20 – Racines du polynômes orthogonal de Legendre de degré 3 et 4

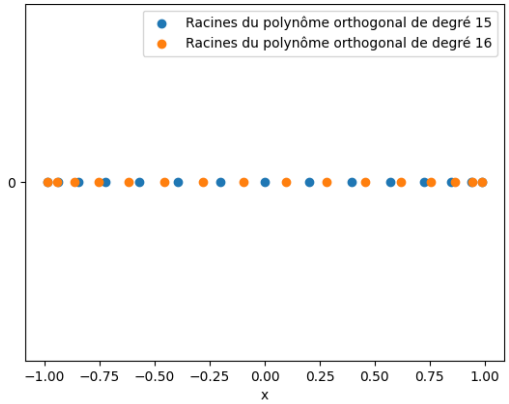


FIGURE 21 – Racines du polynômes orthogonal de Legendre de degré 15 et 16

Ces graphiques mettent bien en évidence la propriété d'entrelacement des racines des polynômes orthogonaux. Cela signifie, en d'autres mots, qu'entre chaque paire de racines consécutives de $L_{n+1}$ se trouve exactement 1 racine de $L_n$.

Passons à présent à la dispersion des racines des polynômes. Voici des histogrammes permettant une bonne visualisation de la situation :

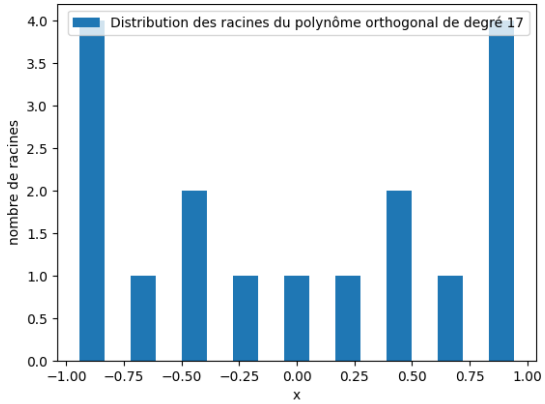


FIGURE 22 – Histogramme des racines du polynôme orthogonal de degré 17

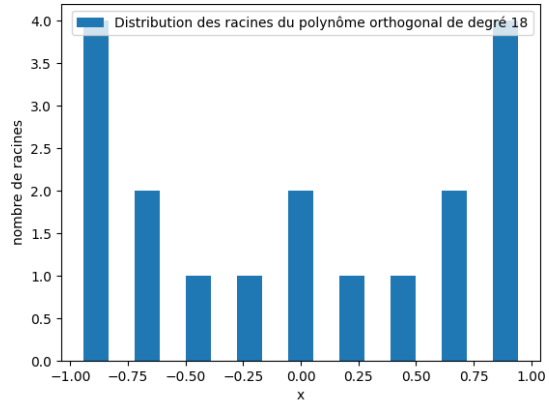


FIGURE 23 – Histogramme des racines du polynôme orthogonal de degré 18

De nouveau, nous pouvons constater que les racines s'accumulent plus sur les bords de l'intervalle $[-1, 1]$. Par ailleurs, on note également une plus légère augmentation au point 0 pour les polynômes de degré pair et à mi-chemin entre le centre et les bords pour les polynômes de degré impair.

# 5 Intégration par quadrature Gaussienne

## 5.1 Les poids de Christoffel

Au TPs, nous avons vu que les $A_{n,k}$, le $k^{ème}$ poids pour une quadrature de degré n, peuvent s'écrire comme

$$A_{n,k} = w_0 O_{1,k}^2$$

où $w_0$ est l'intégrale de la fonction de poids, pour celui de Legendre on a $w_0 = 2$, et où $O_{1,k}$ est la première composante du $k^{ème}$ vecteur propre de la matrice de Jacobi. Cette dernière se construisant grâce aux coefficients de récurrence des polynômes orthogonaux normés de la manière suivante :

$$J_n = \begin{bmatrix} \beta_0 & \alpha_0 & 0 & \cdots & 0 \\ \gamma_1 & \beta_1 & \alpha_1 & \ddots & \vdots \\ 0 & \gamma_2 & \beta_2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \alpha_{n-2} \\ 0 & \cdots & 0 & \gamma_{n-1} & \beta_{n-1} \end{bmatrix}$$

Il nous reste donc à en calculer les vecteurs propres pour connaître les poids de Christoffel grâce aux coefficients trouvés préalablement.

Concernant les poids de Christoffel pour le poids de Legendre, il suffit de calculer les coefficients des polynômes orthogonaux normés associés et d'implémenter ce que l'on vient de décrire.

## 5.2 Quadrature de Gauss-Legendre

La règle de quadrature nous permet d'approximer l'intégrale d'une fonction $I(f) = \int_b^a f(x)dx$ par $I_n(f)$ de la forme

$$I_n(f) = \sum_{j=0}^{n} A_{n,j} f(x_j)$$

pour certains noeuds $-1 \leq x_0 \leq ... \leq x_n \leq 1$ et poids de Christoffel $A_{n,0}, ..., A_{n,n} \in \mathbb{R}$. Celle-ci est dite exacte pour $f$ un polynôme si $I_n(f) = I(f)$. On définira aussi le degré de la quadrature comme le plus grand n tel que cette affirmation reste vraie.

Dans le cadre d'une quadrature de Gauss-Legendre, les noeuds sont donnés par les racines du polynôme orthogonal de Legendre de degré $n+1$. Avec ce choix de noeuds de Gauss, on peut vérifier que le degré de quadrature s'élève à $2n+1$ pour seulement $n+1$ noeuds. De plus, les polynômes de Legendre nous donnent également les poids $A_{n,k}$ calculés dans la sous-section précédente lorsqu'ils sont normés.

Ainsi, pour la définition de la fonction, nous aurons besoin de ces deux éléments : les poids de Christoffel pour les polynômes normés de Legendre et les racines de ces mêmes polynômes. Celles-ci étant déjà implémentées, il reste à sommer leurs produits élément par élément.

## 5.3 Approximation d'intégrale

Appliquons à présent notre précédent travail aux fonctions analysées dans ce projet. Voici les valeurs trouvées pour les degrés de 1 à 10 :

| Degré | Exponentielle | Quotient | Valeur absolue |
|---|---|---|---|
| 1 | 2.3426960879 | 0.2142857142 | 1.1547005383 |
| 2 | 2.3503369286 | 0.9583333333 | 0.8606629658 |
| 3 | 2.3504020921 | 0.3709273182 | 1.0425348572 |
| 4 | 2.3504023864 | 0.7069479203 | 0.9448504363 |
| 5 | 2.3504023872 | 0.4617005584 | 1.0198940935 |
| 6 | 2.3504023872 | 0.6161220802 | 0.9705387022 |
| 7 | 2.3504023872 | 0.5081219176 | 1.0115280634 |
| 8 | 2.3504023872 | 0.5787026971 | 0.9816897567 |
| 9 | 2.3504023872 | 0.5303718848 | 1.0075216905 |
| 10 | 2.3504023872 | 0.5624581121 | 0.9875231094 |

En premier lieu nous pouvons constater que l'exponentielle se stabilise extrêmement rapidement contrairement aux deux autres qui peinent à obtenir 2 décimales exactes. Rappelons que les valeurs à approcher sont :

| Exponentielle | Quotient | Valeur absolue |
|---|---|---|
| 2.3504023872 | 0.5493603067 | 1 |

Ensuite, nous remarquons que pour la fonction quotient nous atteignons des erreurs de presque $\frac{1}{2}$ pour les très petits degrés, contrairement à la valeur absolue qui tourne directement autour de 1. Nous verrons plus loin alors jusqu'à quel degré nous devrions aller pour obtenir au maximum une erreur de $10^{-3}$. Pour ce qui est de la fonction exponentielle, le degré 2 et donc 3 noeuds semblent déjà suffisants. Voyons ce qu'il en est pour le quotient et la valeur absolue.

### 5.3.1 Le quotient

Notre objectif étant de borner l'erreur de l'approximation de la fonction quotient, on utilisera le théorème du cours :

Soit $f \in C[a,b]$,

$$|I_n[f] - I[f]| \leq 2\|w\|_1 \|f - p\|_\infty \qquad \forall p \in \mathcal{P}_{2n+1}$$

On a dans notre cas que $\|w\|_1 = 2$. Pour clarifier les indices, on posera $m = 2n+1$. Ainsi, si nous choisissons les polynômes de Bernstein

$$Bm[f](x) = \sum_{k=0}^{m} f\left(\frac{k}{m}\right) \binom{m}{k} x^k (1-x)^{m-k}$$

Nous avons la borne

$$\|f - Bm[f]\|_\infty \leq \frac{3}{2} w_f \left(\frac{1}{\sqrt{m}}\right)$$

Où $w_f(\delta)$ est le module de continuité évalué en $\delta$. En combinant ces 2 bornes on obtient alors

$$|I_n[f] - I[f]| \leq 6 w_f \left(\frac{1}{\sqrt{m}}\right) \tag{3}$$

Enfin nous cherchons donc $m$ tel que le membre de droite soit inférieur à $10^{-3}$. Essayons dès lors de borner le module de continuité du quotient par une expression dépendant de $n$. On a pour $x, y \in [-1, 1]$

$$\begin{aligned}
w_q(\delta) &= \sup_{|x-y|<\delta} \left| \frac{1}{1+25x^2} - \frac{1}{1+25y^2} \right| \\
&= \sup_{|x-y|<\delta} \left| \frac{25y^2 - 25x^2}{1+25x^2+25y^2+625x^2y^2} \right| \\
&= \sup_{|x-y|<\delta} \left( \frac{625|y-x||y+x|}{1+25x^2+25y^2+625x^2y^2} \right) \\
&\leq \sup_{|x-y|<\delta} \left( \frac{625\delta|y+x|}{1+25x^2+25y^2+625x^2y^2} \right)
\end{aligned}$$

Sur le graphe ci-dessous, nous pouvons voir que $\frac{625|y+x|}{1+25x^2+25y^2+625x^2y^2}$ est borné par 90. Comme cette fonction est continue, nous n'observerons pas de point extrême plus haut que cela. Nous avons alors

$$w_q \left(\frac{1}{\sqrt{m}}\right) \leq \frac{90}{\sqrt{m}}$$

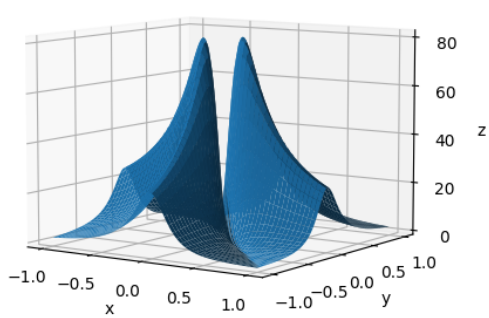


FIGURE 24 – graphe de la fonction $\frac{625|y+x|}{1+25x^2+25y^2+625x^2y^2}$

En conclusion, nous avons l'inégalité

$$|I_n[f] - I[f]| \leq 6\frac{90}{\sqrt{m}} = \frac{540}{\sqrt{m}}$$

Rappelons que $m = 2n + 1$ et nous pouvons ainsi en déduire un n tel que l'erreur sera inférieure à $10^{-3}$, i.e. on voudra $m \geq 540^2 10^6$ et donc $n \geq 145800000000$. Comme nous parlons ici en terme de degré, pour avoir le nombre de noeuds on en ajoutera 1. Ce nombre est remarquablement grand il sera difficile de calculer sans erreur une telle approximation.

### 5.3.2 La valeur absolue

En reprenant la borne explicitée plus tôt (voir (3)), nous devons de nouveau borner le module de continuité pour la valeur absolue cette fois. Cela se fait bien plus facilement, nous avons directement pour $x, y \in [-1, 1]$

$$w_{val}(\delta) = \sup_{|x-y|<\delta} ||x| - |y|| \leq |x - y| \leq \delta = \frac{1}{\sqrt{m}}$$

et donc la borne

$$|I_n[f] - I[f]| \leq \frac{6}{\sqrt{m}}$$

Pour une précision de $10^{-3}$, on voudra dès lors un $m \geq 6 \cdot 10^6$ et donc un $n \geq 3000000$. Encore une fois, il s'agit ici du degré d'approximation, on aura donc 1 noeud de plus pour atteindre la précision tant attendue.

# A Annexe

## A.1 Code Python, fonctions demandées

```python
import math as m
import sympy as sp
import numpy as np
import mpmath as mat
from scipy.optimize import minimize
from numpy.linalg import inv, det
from numpy.linalg import eig

"""-------------------------------------------------------

LMAT1351 : Approximation : méthodes et théorie, projet 2023
Partie code, implémentation des fonctions demandées

-------------------------------------------------------"""

#-----------------------------------------------------

#QUESTION 1 : Interpolation polynomiale
#-----------------------------------------------------


# Arguments: une variable x
# Retrurn: L'évaluation de x en la fonctoin quotient: 1/1+25x**2

def quotient (x):

    return 1 / (1 + 25*x**2)


# Arguments: des coefficients trouvés préalablement, le noeud evalué, la liste des autres noeuds
# Return: l'evaluation de " C_0 + C_1(node - x_0) + C_2(node - x_0)(node - x_1) + ... "

def evaluation (coeff, node, nodesList):

    eval = coeff[0]

    if len(nodesList) > 0:
        eval += (node - nodesList[0]) * evaluation(coeff[1:], node, nodesList[1:])

    return eval



# Arguments: la liste des noeuds, l'indice auquel on est arrivé
# Return: le produit " (x_i - x_0)(x_i - x_1)..."
```

```python
def prod (nodesList, i):

    prod = 1

    for j in range (i):
        prod *= (nodesList[i] - nodesList[j])

    return prod


# Arguments: une liste de noeuds et leur évaluation en la fonction interpolée
# Return: une fonction qui est le polinôme d'interpolation

def interpolation (nodesList, values):

    coeff = []

    for i in range (len(nodesList)):

        if i == 0:
            coeff.append(values[0])

        else:
            coeff.append((values[i] - evaluation(coeff, nodesList[i],
            nodesList[:i-1]))/prod(nodesList, i))

    # Arguments: Une variable x
    # Return: L'évaluation du polynôme calculer en x
    def polynome (x):
        return evaluation(coeff, x, nodesList[:-1])

    return polynome


# Arguments: la liste des noeuds d'interpolation
# Return: la fonction W_n+1 du reste de Lagrange: (x - x0) ... (x - xn)

def w_n1 (nodesList):

    def function (x):
        prod = 1

        for i in range (len(nodesList)):
            prod *= abs(x - nodesList[i])

        return prod

    return function



# Arguments: la liste des noeuds d'interpolation et la fonction souhaitée
#('quotient' pour 1/1+25x**2, 'exp' pour l'exponentielle)
# Return: une fonction de x représentant la borne théorique de du reste de Lagrange
```

```python
def reste_Lagrange (nodesList, function):

    # Ici n représente la taille de la liste des noeuds et donc le degré de l'interpolation + 1
    n = len(nodesList)

    coeff_facto = 1/(m.factorial(n))

    x = sp.symbols('x')

    # Calcule de la norme infini du quotient ou de l'exponentielle
    if function == "quotient":

        # n ème dérivée du quotient
        n1_deriv = sp.diff(-1/(1+25*x**2), x, n)
        f = sp.lambdify(x, n1_deriv)

        # Pour prendre le max on minimise l'opposé, (on a pris l'opposé lors du calcul de la dérivée)
        # Notons que le max se trouve toujours dans [-1, 1]
        # nous n'avons donc pas besoin de nous en occuper
        maximum = - (minimize(f, 0).fun)

    elif function == "exp":
        maximum = m.exp(1)

    else:
        print("Oups ! Try with 'exp' if you want with the exponential or

        'quotient' if you want 1/1+25x**2.")

    w = w_n1(nodesList)

    # fonction évaluant la borne théorique
    def function (x):
        return coeff_facto * maximum * w(x)

    return function



#-----------------------------------------------------

#QUESTION 3 : Splines d'interpolation
#-----------------------------------------------------



# Arguments: une variable x et la liste des noeuds d'interpolation
# Return: L'indice de l'emplacement de x en fonction des noeuds
# (si x < x_j, alors x est à l'indice j)
# Utilisations: dans la définition de la fonction spline
```

```python
def FindIndice (x, nodes):

    for i in range(len(nodes)):
        if x < nodes[i]:
            return i
    return len(nodes)-1


# Arguments: une variable x, les lists des noeuds et des coefficients recherchés z, c et d,
#ainsi que l'indice actuel (S change aux bords)
# Return: La fonction S(x) donnant la spline d'interpolation

def S(x, listx, listz, listc, listd, i):

    # si nous sommes au tout début, on a z_0 = 0
    if i-1 == 0:
        sum = ((x - listx[i-1])**3 / (6*(listx[i] - listx[i-1]))) * listz[i-1]
        sum += listc[i-1]*(x - listx[i - 1]) + listd[i-1]*(listx[i] - x)

    elif i == len(listx)-1:
        sum = ((listx[i] - x)**3 / (6*(listx[i] - listx[i-1]))) * listz[i-2]
        sum += listc[i-1]*(x - listx[i - 1]) + listd[i-1]*(listx[i] - x)

    # A la fin on a z_n = 0
    else:
        sum = ((listx[i] - x)**3 / (6*(listx[i] - listx[i-1]))) * listz[i-2]
        sum += ((x - listx[i-1])**3 / (6*(listx[i] - listx[i-1]))) * listz[i-1]
        sum += listc[i-1]*(x - listx[i - 1]) + listd[i-1]*(listx[i] - x)

    return sum


# Arguments: une variable x, les lists des noeuds et des coefficients recherchés z, c et d,
# ainsi que l'indice actuel (S change aux bords)
# Return: La dérivée en x de la fonction S(x) donnant la spline d'interpolation

def Sprim(x, listx, listz, listc, listd, i):

    # si nous sommes au tout début, on a z_0 = 0
    if i-1 == 0:
        sum = (listz[i-1]*(x - listx[i-1])**2) / (2*(listx[i] - listx[i-1]))
        sum += listc[i-1] - listd[i-1]

    elif i == len(listx)-1:
        sum = (-listz[i-2]*(listx[i] - x)**2) / (2*(listx[i] - listx[i-1]))
        sum += listc[i-1] - listd[i-1]

    # A la fin on a z_n = 0
    else:
        sum = (-listz[i-2]*(listx[i] - x)**2) / (2*(listx[i] - listx[i-1]))
        sum += (listz[i-1]*(x - listx[i-1])**2) / (2*(listx[i] - listx[i-1]))
        sum += listc[i-1] - listd[i-1]

    return sum
```

```python
# Arguments: la liste des noeuds d'interpolation ainsi que leur évaluation en la fonction interpolée
# Return: Une fonction de x donnant la spline d'interpolation

def splineinterpol(nodes, values):

    # Initialisation des variables/symbols sympy
    z_var = []
    c_var = []
    d_var = []

    for i in range(1, len(nodes)):
        if i != len(nodes)-1:
            z_var.append('z'+str(i))
        c_var.append('c'+str(i))
        d_var.append('d'+str(i))

    listz = sp.symbols(z_var)
    listc = sp.symbols(c_var)
    listd = sp.symbols(d_var)

    # mise en équation comme décrit dans le rapport
    equations = []

    # condittions S_(x_j) et S+(x_j) = f(x_j)
    equations.append(S(nodes[0], nodes, listz, listc, listd, 1) - values[0])

    for i in range (1, len(nodes)-1):
        equations.append(S(nodes[i], nodes, listz, listc, listd, i) - values[i])
        equations.append(S(nodes[i], nodes, listz, listc, listd, i+1) - values[i])

    equations.append(S(nodes[len(nodes)-1], nodes, listz, listc, listd, len(nodes)-1)
    - values[len(nodes)-1])

    # conditions S_'(x_j) = S+'(x_j)
    for i in range (1, len(nodes)-1):
        equations.append(Sprim(nodes[i], nodes, listz, listc, listd, i)
        - Sprim(nodes[i], nodes, listz, listc, listd, i+1))

    solutions = sp.solve(equations, dict = True)

    # Arguments: une variable x
    # Return: l'évaluation de x en la spline d'interpolation
    def spline(x):
        j = FindIndice(x, nodes)
        coeff_c = []
        coeff_d  =[]
        coeff_z = []

        for k in range (len(nodes)-1):
            coeff_c.append(solutions[0][listc[k]])
            coeff_d.append(solutions[0][listd[k]])
```

```
            if k != len(nodes)-2:
                coeff_z.append(solutions[0][listz[k]])

        return S(x, nodes, coeff_z, coeff_c, coeff_d, j)

    return spline



#-----------------------------------------------------

#QUESTION 4 : Polynômes orthogonaux
#-----------------------------------------------------
#
# Dans cette section, tout les polynômes sont représentés par la liste des coefficients
# du plus haut degré au plus petit


# Arguments: 2 listes, où list1 est plus grande que list2 (ou égales)
# Return: la soustraction éléments par éléments de list1 et list2 en partant de la droite
#       c'est à dire: l1 = [a, b, c, d] - l2 = [e, f, g] == [a, b-e, c-f, d-g]
# Utilisations: Dans la construction du polynôme avec les coefficients de récurrence

def list_soustraction (list1, list2):

    newlist = list1
    difference = len(list1) - len(list2)

    for i in range (len(list2)):
        newlist[difference + i] -= list2[i]

    return newlist


# Arguments: Une list et un scalaire
# Return: La liste dont tout les éléments ont été multipliés par x
# Utilisations: Dans la normalisation des polynômes
#            + dans la construction du polynôme avec les coefficients de récurrence

def list_multiply_by_scalar (list, x):

    newlist = []

    for i in range(len(list)):
        newlist.append(x*list[i])

    return newlist


# Arguments: 2 polynômes
# Return: Une liste des coefficients du produit de ces polynômes
# Utilisations: Dans la construction des coefficients de récurrence
#              + dans la construction du polynôme avec les coefficients de récurrence
```

```python
def multiply_polynome (polynome1, polynome2):

    product = [0]*(len(polynome1)+len(polynome2)-1)

    for i in range(len(polynome1)):
        for j in range(len(polynome2)):
            product[i+j] += polynome1[i]*polynome2[j]

    return product


# Arguments: les coefficients d'un polynôme
# Return: L'évaluation de son intégrale sur l'intervalle [-1, 1] (poids de Legendre)
# Utilisations: Dans la normalisation des polynômes
#               + dans la construction des coefficients de récurrence

def integrale_poids_Legendre(coeff):

    # Travaillant avec des polynômes,
    # on peut calculer explicitement leur intégrale sur [-1, 1] par une somme
    integrale = 0

    for i in range(len(coeff)):
        integrale += (coeff[i] * 2 * ((len(coeff)-i)%2)) / (len(coeff)-i)

    return integrale


# Arguments: n le nombre de coefficients du polynôme recherché
# et un polynôme connu de degré inférieur ou égale à n
# Return: Une liste de coefficients repésentant ceux de l'équation recherchée.

# Cette équation est définie en réalisant le produit du p_n inconnu avec polynome
# et en intègrant le tout sur l'intervalle [-1, 1]
# les inconnues sont donc les coefficients de p_n.

def mise_en_equation (polynome, n):

    # de nouveau, travaillant avec des polynômes, on peut calculer leur intégrale explicitement.
    # Dans ce cas on en extrait une expression linéaire des coefficients du polynôme inconnu
    coeffs = [0]*n

    for i in range(len(polynome)):
        for j in range(n):
            coeffs[j] += 2 * polynome[i] * 1/(len(polynome)-i + n-j-1) *((len(polynome)-i + n-j-1)%2)

    return np.array(coeffs)


# Arguments: n le nombre de coefficients du polynôme recherché
# et une liste de polynômes connu de degré inférieur ou égale à n
# Return: Une matrice et un vecteur donnant un système linéaire avec toutes les équations
# interprétant l'orthogonalité des polynômes de la liste avec p_n,
# plus l'équation représentant la condition L(1) = 1
```

```python
def systeme_lineaire (list_of_polynomes, n):

    matrice = []

    # Equations représentant les produits scalaires
    for i in list_of_polynomes:
        matrice.append(mise_en_equation(i, n))

    # Evaluation du polynôme en 1
    matrice.append(np.array([1]*n))

    # Définition du vecteur, on veut que leurs produits scalaires valent 0 et que L(1) = 1
    vecteur = [0]*n
    vecteur[-1] = 1

    return (np.array(matrice), np.array(vecteur))


# Arguments: n le degré du polynôme orthogonal tel que L(1) = 1
# Return: Le polynôme orthogonal recherché

def polynome_orthogonaux (n):

    if n == -1:
        return [0]

    elif n == 0:
        return [1]

    else:

        # Défini par récurrence, il nous faut calculer les n-1 premiers polynômes d'abord
        list_of_polynomes = []

        for i in range (n):
            list_of_polynomes.append(polynome_orthogonaux(i))

        # On réalise le système linéaire puis on le résoud
        systeme = systeme_lineaire(list_of_polynomes, n+1)
        solution = np.linalg.solve(systeme[0], systeme[1])

        return list(solution)


# Arguments: n le degré du polynôme orthogonal normé
# Return: Le polynôme orthogonal normé recherché

def polynome_orthonormaux (n):

    pn = polynome_orthogonaux(n)

    # Calcule de la norme
    norme_pn = m.sqrt(integrale_poids_Legendre(multiply_polynome(pn, pn)))
```

```python
    # Si celle-ci vaut 0 on évite une erreur 'division by 0'
    if norme_pn != 0:
        return list_multiply_by_scalar(pn, 1/norme_pn)

    else:
        return pn


# Arguments: n le nombre de coefficients de récurrence voulu,
# normaux = True pour les coefficients dans le cas normaux
# Return: Un triplet de listes des coefficients de récurrence alpha, beta, gamma dans cet ordre

def coefficient_recurrence_Legendre (n, normaux = False):

    alpha = []
    beta = []
    gamma = []

    for i in range(n):

        # Si on veut les coefficients de récurrence pour les polynômes normés
        if normaux:
            pn_1 = polynome_orthonormaux(i-1)
            pn = polynome_orthonormaux(i)
            pn1 = polynome_orthonormaux(i+1)

            x_pn = multiply_polynome(([1, 0]), pn)

            prod_n_1 = multiply_polynome(x_pn, pn_1)
            prod_n = multiply_polynome(x_pn, pn)
            prod_n1 = multiply_polynome(x_pn, pn1)

            # les coeficients sont définits par les produits scalaires
            # gamma : <x_pn, pn-1>
            gamma.append(integrale_poids_Legendre(prod_n_1))
            # beta : <x_pn, pn>
            beta.append(integrale_poids_Legendre(prod_n))
            # alpha : <x_pn, pn+1>
            alpha.append(integrale_poids_Legendre(prod_n1))

        # Pour ceux-ci, les détails sont écrits dans le rapport
        else:
            pn = polynome_orthogonaux(i)
            pn_1 = polynome_orthogonaux(i-1)

            x_pn = multiply_polynome(([1, 0]), pn)

            prod_n = multiply_polynome(x_pn, pn)
            prod_n_1 = multiply_polynome(x_pn, pn_1)

            norme_carre_pn = integrale_poids_Legendre(multiply_polynome(pn, pn))
            norme_carre_pn_1 = integrale_poids_Legendre(multiply_polynome(pn_1, pn_1))
```

```
            # si la norme vaut 0, on évite une erreur 'division by 0'
            if norme_carre_pn_1 != 0:
                gamma.append(integrale_poids_Legendre(prod_n_1)/norme_carre_pn_1)

            else:
                gamma.append(integrale_poids_Legendre(prod_n_1))

            beta.append(integrale_poids_Legendre(prod_n)/norme_carre_pn)

            # On évite ici des calcules inutiles, comme on sait que pn(1) = 1 pour tout n,
            # on a que            1 = a_n + b_n + g_n
            # ce qui implique     a_n = 1 - b_n - g_n
            alpha.append(1 - beta[i] - gamma[i])

    return alpha, beta, gamma


# Arguments: Les coefficients de récurrence alpha, beta, gamma
# Return: Une liste des coefficients du polynôme construit grâce aux coefficients de récurrence

def coeffs_polynome_by_coeffs (alpha, beta, gamma):

    # Application de la formule de récurrence donnée dans le cours: x pn = a pn+1 + b pn + g pn-1

    if len(alpha) -1 == -1:
        return [0]

    elif len(alpha) == 1:
        return [1]

    else:
        pn = coeffs_polynome_by_coeffs(alpha[:-1], beta[:-1], gamma[:-1])
        pn_1 = coeffs_polynome_by_coeffs(alpha[:-2], beta[:-2], gamma[:-2])

        x_pn = multiply_polynome([1, 0], pn)
        beta_n_pn = list_multiply_by_scalar(pn, beta[-1])
        gamma_n_pn_1 = list_multiply_by_scalar(pn_1, gamma[-1])

        x_beta_n_pn = list_soustraction(x_pn, beta_n_pn)
        x_beta_n_pn_gamma_n_pn = list_soustraction(x_beta_n_pn, gamma_n_pn_1)

        return list_multiply_by_scalar(x_beta_n_pn_gamma_n_pn, 1/alpha[-1])


# Arguments: Une liste de coefficients
# Return: Une fonction évaluant le polynôme donné par les coefficients

def make_function_with_coeffs (coeffs):

    def polynome (x):

        value = 0
        for i in range(len(coeffs)):
            value += coeffs[i]*(x**(len(coeffs)-i-1))
```

```
        return value

    return polynome


# Arguments: Les coefficients de récurrence alpha, beta, gamma
# Return: Une fonction évaluant le polynôme construit sur base des coefficients de récurrence

def polynome_made_by_coeffs_recurrence (alpha, beta, gamma):

    coeffs = coeffs_polynome_by_coeffs(alpha, beta, gamma)

    return make_function_with_coeffs(coeffs)


# Arguments: n le degré du polynôme dont on veut les racines
# Return: Une liste triée des racines du polynôme voulu

def racine_polynome_legendre (n):

    coeffs = polynome_orthogonaux(n)

    roots = mat.polyroots(coeffs, maxsteps = 1000)

    return roots


#-----------------------------------------------------

#QUESTION 5 :  Intégration par quadrature Gaussienne
#-----------------------------------------------------


# Arguments: une liste et une matrice
# Return: La liste triée et la matrice permutée
# de sorte que les lignes correspondent toujours aux éléments de la liste

def my_sort(liste, matrice):
    # Créer une liste d'indices triés
    indices_tries = sorted(range(len(liste)), key=lambda k: liste[k])

    # Trier la liste
    liste_triee = [liste[i] for i in indices_tries]

    # Permuter les lignes de la matrice
    matrice_permutee = [matrice[i] for i in indices_tries]

    return liste_triee, matrice_permutee


# Arguments: les coefficients de récurrence alpha, beta, gamma
# Return: La matrice de Jacobi qui est tridiagonale avec alpha sur la diagonale supérieure,
#         beta sur la diagonale et gamma sur la diagonale inférieure
```

```python
def matrice_jacobi(alpha, beta, gamma):
    n = len(alpha)

    matrice = []

    for i in range(n):
        matrice.append([0]*n)

        for j in range(n):
            if i+1 == j:
                matrice[i][j] += alpha[i]
            elif i == j:
                matrice[i][j] += beta[i]
            elif i-1 == j:
                matrice[i][j] += gamma[i]

    return matrice


# Arguments: n tel que n-1 est le degré de la quadrature (commence à 0)
# et un triplet de listes des coefficients de récurrence
# Return: Une liste avec les poids de Christoffel

def Ak(n, coeffs):

    # calcules des valeurs et vecteurs propres
    matrix = np.array(matrice_jacobi(coeffs[0], coeffs[1], coeffs[2]))
    eigen = np.linalg.eig(matrix)

    # Tri des valeurs propres dans l'ordre croissant
    # en gardant la correspondance avec les vecteurs propres
    vect_propres = my_sort(eigen[0], np.transpose(eigen[1]))[1]

    list_Ak = []

    for i in range(n):
        list_Ak.append(2*(vect_propres[i][0])**2)

    return list_Ak


# Arguments: n tel que n-1 est le degré de la quadrature
# Return: la liste des poids de Christoffel pour les coefficients de récurrence de Legendre

def Ak_legendre (n):

    coeffs = coefficient_recurrence_Legendre(n, normaux = True)

    return Ak(n, coeffs)
```

```python
# Arguments: Une fonction f que l'on veut intégrer
# et n le degré de la quadrature (le bon cette fois)
# Return: L'approximation de l'intégrale de f

def approximation_integrale(f, n):

    integrale = 0
    roots = racine_polynome_legendre(n+1)
    Ak = Ak_legendre(n+1)

    for i in range(len(roots)):
        integrale += Ak[i]*f(roots[i])

    return integrale
```

## A.2 Code Python, graphes

```python
import numpy as np
import matplotlib.pyplot as mp
import LMAT1351_code_fonctions as p
import math as m

"""-------------------------------------------------------

LMAT1351 : Approximation : méthodes et théorie, projet 2023
Partie code, implémentation des fonctions des graphes

-------------------------------------------------------"""

#-----------------------------------------------------

#QUESTION 1 : Interpolation polynomiale
#-----------------------------------------------------


# Argument: entrée x
# Return: l'évaluation de la fonction 1 / 1+25x**2

def quotient (x):
    return 1 / (1 + 25*x**2)


# Arguments: une fonction de x, liste des degrés d'interpolation désirés
# Affiche: UN graph avec les interpolations de la fonction donnée pour les degrés demandés,
# ansi que la fonction initiale

def graph_interpolation (funct, list_degre):

    figure, axes = mp.subplots()
    axes.set_xlabel("x")
    axes.set_ylabel("y")

    x1 = np.linspace(-1, 1, 1000)
    y1 = []
    for i in x1:
        y1.append(funct(i))

    axes.plot(x1, y1, color = 'red', label = "fonction interpolée")

    for j in list_degre:

        if j == 0:
            nodesList = [0]

        else:
            nodesList = np.linspace(-1, 1, j+1)

        values = []
```

```
        for i in nodesList:
            values.append(funct(i))

        y2 = []
        for k in x1:
            y2.append(p.interpolation(nodesList, values)(k))

        axes.plot(x1, y2, label = str(j+1)+" noeuds")

        y2.clear()
        values.clear()

    axes.legend()
    mp.show()


# Arguments: le nombre de noeuds désiré, la fonction
# ('quotient' pour le quotient et 'exp' pour l'exponentielle)
# difference = False si on ne veut pas montrer la distance entre l'interpolation et la fonction
# reste = False si on ne veut pas montrer le graphe du reste de Lagrange
# Affiche: Un graphe avec la distance entre l'interpolation et la fonction plus le reste de Lagrange

def comparaison_reste_Lagrange (nbr, fonction, difference = True, reste = True,):

    figure, axes = mp.subplots()
    axes.set_xlabel("x")
    axes.set_ylabel("y")

    nodesList = np.linspace(-1, 1, nbr)
    if difference:
        x1 = np.linspace(-1, 1, 10000)

        # construction des noeuds et de leurs évaluations
        values = []
        for i in nodesList:
            if fonction == "quotient":
                values.append(quotient(i))
            elif fonction == "exp":
                values.append(m.exp(i))
            else:
                print("Oups ! Try with 'exp' if you want with the exponential
                or 'quotient' if you want 1/1+25x**2.")
                break

        # différence entre la fonction interpolée et son interpolation
        y1 = []
        for k in x1:
            if fonction == "quotient":
                y1.append(abs(quotient(k) - p.interpolation(nodesList, values)(k)))
            elif fonction == "exp":
                y1.append(abs(m.exp(k) - p.interpolation(nodesList, values)(k)))
            else:
                break
```

```
        axes.plot(x1, y1, color = 'red', label = "distance à la fonction interpolée")

    if reste:
        x2 = np.linspace(-1, 1, 1000)

        reste_Lagrange = p.reste_Lagrange(nodesList, fonction)

        y2 = []
        for i in x2:
            y2.append(reste_Lagrange(i))

        axes.plot(x2, y2, color = 'blue', label = "reste de Lagrange")

    axes.legend()
    mp.show()


#-----------------------------------------------------


#QUESTION 3 : Splines d'interpolation
#-----------------------------------------------------


# Arguments: La fonction à interpolée ainsi que la liste des nombres de noeuds désirés
# ATTENTION: minimum 3 noeuds
# Affiche: Le graphe des splines d'interpolation constuites sur les nombres de noeuds voulus,
# ainsi que la fonction initiale

def graph_spline(funct, list_nbr_noeuds):

    figure, axes = mp.subplots()
    axes.set_xlabel("x")
    axes.set_ylabel("y")

    x1 = np.linspace(-1, 1, 1000)
    y1 = []
    for i in x1:
        y1.append(funct(i))

    axes.plot(x1, y1, color = 'red', label = "fonction interpolée")

    for j in list_nbr_noeuds:
        if j == 1:
            list = [0]

        else:
            list = np.linspace(-1, 1, j)

        values = []
        for i in list:
            values.append(funct(i))

        spline = p.splineinterpol(list, values)
```

```
        x = np.linspace(-1, 1, 1000)
        y = []

        for i in x:
            y.append(spline(i))

        axes.plot(x, y, label = "spline à "+str(j)+ " noeuds")

    axes.legend()
    mp.show()

def f (x):
    return m.exp(x)


#-----------------------------------------------------

#QUESTION 4 : Polynômes orthogonaux
#-----------------------------------------------------


# Arguments: Une liste des degrés des polynômes que l'on souhaite afficher
# Affiche: Le polynôme orthogonale tel que L(1) = 1 de degré n

def graph_polynome_legendre (list_degre):

    figure, axes = mp.subplots()
    axes.set_xlabel("x")
    axes.set_ylabel("y")

    x = np.linspace(-1, 1, 100)

    for i in list_degre:

        function = p.make_function_with_coeffs(p.polynome_orthogonaux(i))

        y = function(x)

        axes.plot(x, y, label = "Polynôme orthogonal de Legendre de degré " + str(i))

    axes.legend()
    mp.show()


# Arguments: n le degré du polynôme dont on calcule les racines ATTENTION: degré max = 18,
# sinon cela ne converge pas assez rapidement dans le calcule des racines (1000 steps)
# hist = True si on veut montrer l'histograme
# subdivsion est le nombre de batonnet de l'histogramme
# double = True si on veut montrer les racines du polynômes de degré n+1 en plus (entrelacement)
#
# Affiche: Soit les racines du polynômes de dergé n
#          Soit l'histogramme des racines du polynôme de Legendre avec 'subdivision' batonnets
#          Soit les racines des polynômes de degrés n et n+1
```

```python
def graph_racine_polynome_legendre(n, hist = False, subdivision = 9, double = False):


    figure, axes = mp.subplots()

    x = p.racine_polynome_legendre(n)
    y = [0]*n

    if hist:
        axes.set_xlabel("x")
        axes.set_ylabel("nombre de racines")
        axes.hist(x, range = (-1,1), bins = subdivision, rwidth = 0.5,
        label = "Distribution des racines du polynôme orthogonal de degré "+str(n))

    elif double:
        axes.set_xlabel("x")
        mp.yticks([0])
        x2 = p.racine_polynome_legendre(n+1)
        y2 = [0]*(n+1)
        axes.scatter(x, y, label = "Racines du polynôme orthogonal de degré "+str(n))
        axes.scatter(x2, y2, label = "Racines du polynôme orthogonal de degré "+str(n+1))

    else:
        axes.set_xlabel("x")
        mp.yticks([0])
        axes.scatter(x, y)

    axes.legend()
    mp.show()


# Arguments: /
# Affiche: Le graphe en 3D de 625|X+Y| / 1 + 25(X**2) + 25(Y**2) + 625(X**2)*(Y**2)

def borne_module_quotient():


    fig = mp.figure()
    ax = mp.axes(projection='3d')

    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")

    x = np.linspace(-1, 1, 1000)
    y = np.linspace(-1, 1, 1000)
    X, Y = np.meshgrid(x, y)
    Z = 625*(abs(X+Y))/(1 + 25*(X**2) + 25*(Y**2) + 625*(X**2)*(Y**2))

    ax.plot_surface(X, Y, Z)

    mp.show()
```

## A.3 Bibliographie


1. Syllabus du cours
2. ChatGPT (en tant que aide informatique pour résoudre des problèmes mineurs)
3. Documentation Sympy, Numpy, Matplotlib.pyplot, etc
4. https ://pomax.github.io/bezierinfo/legendre-gauss.html - pour vérifier les poids de Christoffel et les racines des polynômes
5. https ://fr.wikipedia.org/wiki/Polynôme_de_Legendre - pour vérifier les coefficients des polynômes orthogonaux de Legendre